

02393 Programming in C++

Module 5: Libraries and Interfaces

© **Giovanni Meroni**

Slides based on previous versions by Alceste Scalas, Andrea Vandin, Alberto Lluch Lafuente, Sebastian Mödersheim

Lecture plan

Module	Date	Title	Textbook chapters
0 and 1	03/09	Welcome & C++ Overview	1
2	10/09	Basics of C++ and its Data Types	1.5, 2.1, 2.3 - 2.5, 11.1, 11.3
3	17/09	Memory management	12.1, 11.2, 11.3
4	24/09	LAB DAY	N/A
5	01/10	Libraries and Interfaces	2.2, 2.6 - 2.8, 3.1 - 3.3, 4.1 - 4.3
6	08/10	Classes and Objects	5.1, 6.1, 11.2, 12.1, 12.4, 12.7
Autumn break			
7	22/10	Templates	5.1, 14.2
8	29/10	LAB DAY + Guest lecture	N/A
9	05/11	Inheritance	19.1 - 19.3
10	12/11	Recursive Programming	7
11	19/11	Linked Lists	12.2, 12.3
12	26/11	Trees	16.1 - 16.4
13	03/12	LAB DAY	N/A

Outline

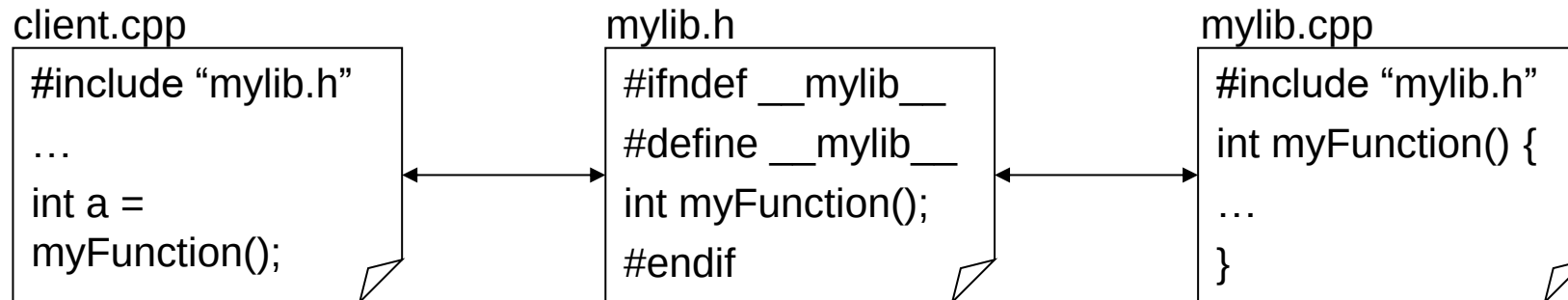
- Recap
- Libraries and interfaces
- The C++ Standard Template Library
- STL Vectors
- Standard I/O and file streams
- STL Strings
- Lab

A recap from the last lectures lecture

- The structure of a C++ program
 - `#include` and `#define` directives, the main function, user-defined functions
- Simple input/output
 - `cin`, `cout`
- Variables, values and types
 - `string`, `int`, `double`, `float`, `bool`, arrays (statically and dynamically allocated), pointers, `enum`, `struct`
- Expressions
 - some numeric and boolean operators and math functions
- Statements
 - `if`, `while`, `for`, `switch`

Libraries and Interfaces

- A library is a collection of ready-to-use type definitions and functions
- A library interface is a “contract” between a library and its users (i.e., us programmers):
 - It provides information for using the library
 - It hides the implementation details
- The interface of a library is a header file (example: `iostream.h`) that only contains:
 - Function prototypes (not implementations!)
 - Type definitions
- The implementation of a library is written in one or more `.cpp` files



Libraries and Interfaces

```
1 #include <iostream>
2 #include "mylib.h"
```

- The `#include` directive is used to specify which libraries are used by a client program

Libraries and Interfaces

```
1 #include <iostream>  
2 #include "mylib.h"
```

- The `#include` directive is used to specify which libraries are used by a client program
 - If the library is part of the STL (and of the compiler), its name of the library is put inside angle brackets

Libraries and Interfaces

```
1 #include <iostream>
2 #include "mylib.h"
```

- The `#include` directive is used to specify which libraries are used by a client program
 - If the library is part of the STL (and of the compiler), its name of the library is put inside angle brackets
 - If the library is NOT part of the STL (e.g., it is created by the developer), the name of its header file is put inside quotes, and it must include the `.h` extension

Libraries and Interfaces

mylib.h

```
1  #ifndef __mylib__
2  #define __mylib__
3
4  int myFunction();
5
6  #endif
```

- To avoid a library to be included multiple times in a program, the `#ifndef...#endif` precompiler directives can be used in the header file.
- `#ifndef` checks if the symbol `__mylib__` is already defined
 - If `mylib` has not been included, then `__mylib__` is not defined. Thus, `mylib` code is included. Also, `__mylib__` is defined.
 - If `mylib` has already been included, then `__mylib__` is defined. Thus, the code block until `#endif` is ignored.

Libraries and Interfaces

- To compile a program that uses libraries not part of the STL, one must pass to the compiler the name of all .cpp files
 - E.g., `g++ client.cpp mylib.cpp -o client.exe`

The C++ Standard Template Library (STL)

- STL is a C++ library of container classes and algorithms
- Containers are collections of elements. Examples:
 - Unordered collections: set, mset
 - Array-like collections: vector, list, array (not the built-in arrays we already know!)
 - Other ordered collections: queue, stack
 - Dictionaries: map, multimap
- It is important to know how to deal with STL containers and choose the right one
 - More than one class of containers may do the job...
 - ...but some may do the job better (e.g., faster)
 - Before writing new code, it may be worth checking whether the STL can help!

STL Vector

- Arrays are fundamental data types in many programming languages, but in C++. . .
 - They are difficult/impossible to resize
 - Insertion and deletion can be difficult and slow
 - You have to keep track of their actual size
 - You have to be careful to index within the array bounds
 - You have to manually handle their allocation on the stack or on the heap
- The STL vector class solves all these problems!

STL Vector

- To use STL vectors, you need to include their interface (header file): `#include <vector>`
- The type `vector<X>` is a container of elements of base type `X`
 - `vector<int>` is a vector whose elements are integers
 - `vector<double>` is a vector whose elements are doubles
 - `vector<vector<int>>` is a vector whose elements are vectors of integers
 - ...and so on

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```

10

- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```

10

20

- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```

10	20	30
----	----	----

- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```

- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1

10	20	30	40
----	----	----	----

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```



- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1
- begin() returns an iterator pointing to the beginning of the array
 - Iterators are a bit like pointers

STL Vector

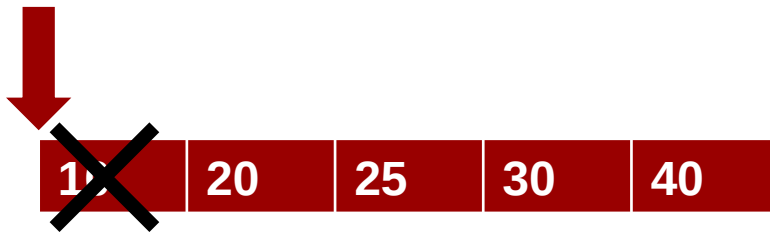
```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```



- push_back(x) adds x at the end of the vector
 - The size of the vector increases by 1
- begin() returns an iterator pointing to the beginning of the array
 - Iterators are a bit like pointers
- insert(it, y) adds y before the element at the position specified by the iterator it
 - The size of the vector increases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```



- `push_back(x)` adds `x` at the end of the vector
 - The size of the vector increases by 1
- `begin()` returns an iterator pointing to the beginning of the array
 - Iterators are a bit like pointers
- `insert(it, y)` adds `y` before the element at the position specified by the iterator `it`
 - The size of the vector increases by 1
- `erase(it)` removes the element at the position specified by the iterator `it`
 - The size of the vector decreases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```

20	25	30	40
----	----	----	----

- `push_back(x)` adds `x` at the end of the vector
 - The size of the vector increases by 1
- `begin()` returns an iterator pointing to the beginning of the array
 - Iterators are a bit like pointers
- `insert(it, y)` adds `y` before the element at the position specified by the iterator `it`
 - The size of the vector increases by 1
- `erase(it)` removes the element at the position specified by the iterator `it`
 - The size of the vector decreases by 1

STL Vector

```
1 vector<int> vec;  
2 vec.push_back(10);  
3 vec.push_back(20);  
4 vec.push_back(30);  
5 vec.push_back(40);  
6 vector<int>::iterator it = vec.begin();  
7 vec.insert(it+2, 25);  
8 vec.erase(it);  
9 vec[3] = 35;
```



20	25	30	35
----	----	----	----

- `push_back(x)` adds `x` at the end of the vector
 - The size of the vector increases by 1
- `begin()` returns an iterator pointing to the beginning of the array
 - Iterators are a bit like pointers
- `insert(it, y)` adds `y` before the element at the position specified by the iterator `it`
 - The size of the vector increases by 1
- `erase(it)` removes the element at the position specified by the iterator `it`
 - The size of the vector decreases by 1
- The content of a vector is indexed and can be accessed with the same syntax as arrays

Iterating through vector elements

```
1 for (int i = 0; i < vec.size(); i++) {  
2     cout << vec[i] << " ";  
3 }
```

- Array-style

```
1 vector<int>::iterator it;  
2 for (it = vec.begin(); it != vec.end(); it++) {  
3     cout << *it << " ";  
4 }
```

- Using iterators

```
1 for (auto e : vec) {  
2     cout << e << " ";  
3 }
```

- Modern style (range-based loop)

STL Vector and memory allocation

```
1 vector<int> makeVector() {  
2     vector<int> result;  
3     // ...  
4     return result;  
5 }  
6  
7 int main () {  
8     vector<int> vec = makeVector();  
9     // 'vec' is used here  
10    return 0;  
11 }
```

- Does this code work? Could it make the program crash?

STL Vector and memory allocation

```
1 vector<int> makeVector() {  
2     vector<int> result;  
3     // ...  
4     return result;  
5 }  
6  
7 int main () {  
8     vector<int> vec = makeVector();  
9     // 'vec' is used here  
10    return 0;  
11 }
```

- The vector internally stores its data in a dynamically allocated array (using new)
 - Hence, this internal array resides on the heap (not on the stack!)
- Some internal information of the vector (pointer to array, size) is kept on the stack
 - Such pointer and size are copied to the caller, inside vec, when makeVector() returns
 - Since the internal array survives on the heap, the code above works!
- When main() ends, vec is destroyed and it automatically deletes its internal array
 - Hence, no need to manually free the memory

STL Vector and memory allocation

```
1 void extendVector(vector<int> v) {  
2     v.push_back(42);  
3 }  
4  
5 int main () {  
6     vector<int> vec;  
7     extendVector(vec);  
8 }
```

- Does this code change vec?

STL Vector and memory allocation

```
1 void extendVector(vector<int> v) {  
2     v.push_back(42);  
3 }  
4  
5 int main () {  
6     vector<int> vec;  
7     extendVector(vec);  
8 }
```

- No: vec is copied when passed to extendVector() (call-by-value, as usual)

STL Vector and memory allocation

```
1 void extendVector(vector<int> &v) {  
2     v.push_back(42);  
3 }  
4  
5 int main () {  
6     vector<int> vec;  
7     extendVector(vec);  
8 }
```

- No: vec is copied when passed to extendVector() (call-by-value, as usual)
- If we want to modify vec, we can use references

STL Vector and memory allocation

```
1 void extendVector(const vector<int> &v) {  
2     v.push_back(42);  
3 }  
4  
5 int main () {  
6     vector<int> vec;  
7     extendVector(vec);  
8 }
```

- No: vec is copied when passed to extendVector() (call-by-value, as usual)
- If we want to modify vec, we can use references
- If we want to ensure that the vector is not modified, we can use the const keyword
 - The function treats the argument as a reference to a constant

STL Vector and memory allocation

```
1 void extendVector(const vector<int> &v) {  
2     v.push_back(42);  
3 }  
4  
5 int main () {  
6     vector<int> vec;  
7     extendVector(vec);  
8 }
```

- No: vec is copied when passed to extendVector() (call-by-value, as usual)
- If we want to modify vec, we can use references
- If we want to ensure that the vector is not modified, we can use the const keyword
 - The function treats the argument as a reference to a constant
 - If the code in the function tries to change vec, a compile error is produced

STL Vector and memory allocation

- Memory handling in STL vectors makes life easier
 - We can often avoid working with pointers, new and delete!
- Other STL containers (set, map, stack, ...) have the same convenient memory handling
- ...but what is going on behind the scenes in these containers?
 - We will see in the upcoming lectures on object-oriented programming in C++

Standard I/O and file streams

- STL includes the I/O library `iostream`, based on streams
 - The `cout` stream writes output to the console with its insertion operator `<<`
 - The `cin` stream takes input from console with its extraction operator `>>`
- STL also includes a stream-based library for file I/O, called `fstream`
 - `ofstream` objects write output to a file with their insertion operator `<<`
 - `ifstream` objects take input from a file with their extraction operator `>>`

Standard I/O and file streams

```
1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      ifstream infile;
7      ofstream outfile("output.txt");
8
9      infile.open("input.txt");
10     if (infile.fail()) {
11         cout << " ERROR : cannot open the file !" << endl;
12     }
13
14     string x;
15     infile >> x;
16     outfile << "I have read the string : " << x << endl;
17
18     infile.close();
19     outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized

Standard I/O and file streams

```
1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      ifstream infile;
7      ofstream outfile("output.txt");
8
9      infile.open("input.txt");
10     if (infile.fail()) {
11         cout << " ERROR : cannot open the file !" << endl;
12     }
13
14     string x;
15     infile >> x;
16     outfile << "I have read the string : " << x << endl;
17
18     infile.close();
19     outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized
 - outfile is initialized: the file output.txt will be opened by the program

Standard I/O and file streams

```
1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      ifstream infile;
7      ofstream outfile("output.txt");
8
9      infile.open("input.txt")
10     if (infile.fail()) {
11         cout << " ERROR : cannot open the file !" << endl;
12     }
13
14     string x;
15     infile >> x;
16     outfile << "I have read the string : " << x << endl;
17
18     infile.close();
19     outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized
 - outfile is initialized: the file output.txt will be opened by the program
- Opening files (if not already done)
 - infile will use the file input.txt

Standard I/O and file streams

```
1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      ifstream infile;
7      ofstream outfile("output.txt");
8
9      infile.open("input.txt");
10     if (infile.fail()) {
11         cout << " ERROR : cannot open the file !" << endl;
12     }
13
14     string x;
15     infile >> x;
16     outfile << "I have read the string : " << x << endl;
17
18     infile.close();
19     outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized
 - outfile is initialized: the file output.txt will be opened by the program
- Opening files (if not already done)
 - infile will use the file input.txt
 - fail() tells the program if something went wrong when manipulating a file

Standard I/O and file streams

```
1 #include <fstream>
2 #include <iostream>
3 using namespace std;
4
5 int main(){
6     ifstream infile;
7     ofstream outfile("output.txt");
8
9     infile.open("input.txt");
10    if (infile.fail()) {
11        cout << " ERROR : cannot open the file !" << endl;
12    }
13
14    string x;
15    infile >> x;
16    outfile << "I have read the string : " << x << endl;
17
18    infile.close();
19    outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized
 - outfile is initialized: the file output.txt will be opened by the program
- Opening files (if not already done)
 - infile will use the file input.txt
 - fail() tells the program if something went wrong when manipulating a file
- Reading/writing data from/to the open files

Standard I/O and file streams

```
1  #include <fstream>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      ifstream infile;
7      ofstream outfile("output.txt");
8
9      infile.open("input.txt");
10     if (infile.fail()) {
11         cout << " ERROR : cannot open the file !" << endl;
12     }
13
14     string x;
15     infile >> x;
16     outfile << "I have read the string : " << x << endl;
17
18     infile.close();
19     outfile.close();
20 }
```

- Stream variable(s) declaration
 - infile is not initialized
 - outfile is initialized: the file output.txt will be opened by the program
- Opening files (if not already done)
 - infile will use the file input.txt
 - fail() tells the program if something went wrong when manipulating a file
- Reading/writing data from/to the open files
- Closing files when no longer needed
 - This happens automatically when the stream goes out of scope

STL Strings

- In C++, strings are natively represented as arrays of chars with last element 0
 - They have all disadvantages of arrays (difficult to resize, risk of out-of-bound access...)
- The STL `<string>` header file provides the string type that makes life much easier
 - We have already used it!
- Operations on STL strings:
 - Assignment using `=` (makes a new copy)
 - Comparison using `<`, `==`, `>=`, ... (by alphabetical ordering)
 - Concatenation using `+`

STL Strings

```
1  #include <string>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      string str1("Hello World");
7      string str2 = "Hello world";
8
9      cout << "str1 is " << (str1.empty() ? "" : "not ")
10         << "empty" << endl;
11
12     cout << "str2 is " << str2.length()
13         << " characters long" << endl;
14 }
```

- String creation

STL Strings

```
1 #include <string>
2 #include <iostream>
3 using namespace std;
4
5 int main(){
6     string str1("Hello World");
7     string str2 = "Hello world";
8
9     cout << "str1 is " << (str1.empty() ? "" : "not ")
10         << "empty" << endl;
11
12     cout << "str2 is " << str2.length()
13         << " characters long" << endl;
14 }
```

- String creation
- Checking if a string is empty
 - True if empty
 - False otherwise

STL Strings

```
1  #include <string>
2  #include <iostream>
3  using namespace std;
4
5  int main(){
6      string str1("Hello World");
7      string str2 = "Hello world";
8
9      cout << "str1 is " << (str1.empty() ? "" : "not ")
10         << "empty" << endl;
11
12     cout << "str2 is " << str2.length()
13         << " characters long" << endl;
14 }
```

- String creation
- Checking if a string is empty
 - True if empty
 - False otherwise
- Getting the length of a string
- ...but what is going on behind the scenes in STL strings?
 - We will see in the upcoming lectures on object-oriented programming in C++

Lab time

- Today
 - Make sure C++ works on your computer, request help if it doesn't
 - Begin working on Assignment 5
 - Ask questions if something is unclear (including previous assignments)

DTU

